

1 Introduction

Ce rapport présente l'analyse des métriques logicielles d'une application de gestion de bibliothèque. L'objectif est d'évaluer la qualité du code source à l'aide de l'outil CKJM, d'identifier les classes problématiques et de proposer des opérations de refactoring pour améliorer la maintenabilité et la robustesse du code. L'analyse se conclut par une comparaison des métriques avant et après les modifications.

2 Synthèse des Métriques Initiales

L'analyse initiale avec CKJM a produit les métriques suivantes pour chaque classe du projet :

Classe	WMC	DIT	NOC	CBO	RFC	LCOM	Ca	NPM
library.model.Book	7	1	1	1	13	5	3	7
library.model.DigitalBook	2	0	0	1	3	0	1	2
library.model.Lendable	2	1	0	0	2	1	4	2
library.service.Library	5	1	0	2	18	0	1	5
library.service.Customer	3	1	0	1	10	0	0	3
library.app.LibraryApp	2	1	0	4	10	1	0	2

TABLE 1 – Métriques CKJM avant refactoring.

3 Interprétation Critique

L'analyse des métriques initiales révèle plusieurs points d'attention :

- **Classe Book** : Présente une complexité (WMC=7) et un manque de cohésion (LCOM=5) élevés. La cause principale est une méthode de test non liée aux responsabilités de la classe.
- **Classe Library** : A un nombre de réponses possibles (RFC=18) très élevé, suggérant qu'elle possède trop de responsabilités. De plus, son couplage (CBO=2) et sa complexité (WMC=5) sont significatifs.
- **Couplage** : La classe LibraryApp a un CBO de 4, indiquant une forte dépendance aux autres classes, ce qui est attendu pour une classe principale.

4 Propositions de Refactoring

Pour adresser ces problèmes, les trois opérations de refactoring suivantes ont été mises en œuvre :

1. **Améliorer la cohésion de Book** : Suppression de la méthode `complexMethodExample`, qui était une source de complexité inutile et n'avait aucun lien avec la logique métier de la classe.
2. **Utiliser le polymorphisme dans Library** : Remplacement des vérifications `instanceof` par une approche polymorphique. Une méthode `getDescription()` a été ajoutée à l'interface `Lendable` et implémentée dans les classes `Book` et `DigitalBook`. Cela réduit le couplage et simplifie la méthode `listAllItems`.
3. **Corriger la logique métier** : La méthode `getAvailableItems` a été corrigée pour ne retourner que les livres non empruntés, améliorant ainsi la robustesse de l'application.

5 Comparaison Avant / Après Refactoring

Après application des modifications, le code a été recompilé et analysé à nouveau. Le tableau ci-dessous compare les métriques avant et après refactoring.

	Avant Refactoring		Après Refactoring	
Classe	WMC	LCOM	WMC	LCOM
library.model.Book	7	5	3	0
library.model.DigitalBook	2	0	3	1
library.model.Lendable	2	1	3	3
library.service.Library	5	0	5	0

TABLE 2 – Évolution des métriques WMC et LCOM.

5.1 Analyse de l'Évolution

- **Classe Book** : Le changement est spectaculaire. Le WMC a chuté de 7 à 3 et le LCOM de 5 à 0. La classe est maintenant beaucoup plus simple, plus cohérente et respecte le principe de responsabilité unique.
- **Classe DigitalBook et Lendable** : L'ajout de la méthode `getDescription` a légèrement augmenté leur complexité (WMC), ce qui est un compromis acceptable pour le gain obtenu en flexibilité et en réduction du couplage dans la classe `Library`.
- **Classe Library** : Bien que son WMC n'ait pas changé, le refactoring de `listAllItems` a éliminé une mauvaise pratique de conception (`instanceof`), rendant le code plus propre et plus facile à étendre.

6 Conclusion

Le refactoring a eu un impact très positif sur la qualité globale du code. En ciblant les classes avec les métriques les plus problématiques, nous avons réussi à :

- **Améliorer la maintenabilité** : La réduction de la complexité (WMC) et l'amélioration de la cohésion (LCOM) rendent le code plus facile à comprendre, à modifier et à déboguer.
- **Augmenter la robustesse** : La correction des erreurs logiques et l'utilisation de meilleures pratiques de conception (polymorphisme) réduisent les risques de bugs futurs.
- **Favoriser la réutilisabilité** : Des classes plus cohérentes et moins couplées sont plus faciles à réutiliser dans d'autres contextes.

Ce TP démontre l'importance des métriques logicielles comme outil de diagnostic pour guider les efforts de refactoring et améliorer durablement la qualité d'un projet.

Annexes

Code Source

La version finale du code source, après refactoring, est disponible sur GitHub à l'adresse suivante : [abderrahmanBouanani/Code_Metrics_LOC_POO](https://github.com/abderrahmanBouanani/Code_Metrics_LOC_POO)